PHASE 4 · ADVANCED

DAY 25

Databases & Packaging

sqlite3 · SQLAlchemy ORM · pyproject.toml · Docker

2 hrs	3	20	2
SESSION LENGTH	CONCEPT FILES	PRACTICE PROBS	NEW LIBRARIES

Session Overview

Today students move from Python-only logic to **persistent storage**. They'll query a database with raw SQL via **sqlite3**, then rebuild the same operations using the **SQLAlchemy ORM** so they can work with Python objects instead of SQL strings. The session closes with **packaging** — turning a project into an installable tool via **pyproject.toml** and **uv build** — and a first look at **Docker**, which the Capstone (Days 26-30) will deploy with.

Goal

By the end of this session, students can create and query a SQLite database with sqlite3, define SQLAlchemy ORM models with relationships, perform CRUD through Session objects, and understand what pyproject.toml and a Dockerfile are for.

Outcome

Students leave with three runnable concept files, a 20-problem practice set, and a mental model of the full path from "data in a Python variable" to "data in a deployable, containerized application" — exactly what the Capstone project needs starting Day 26.

Session Timeline

Time	Segment	Focus
0:00 - 0:25	SQLite Basics	sqlite3 connect, CREATE TABLE, CRUD, JOINs
0:25 - 0:55	SQLAlchemy ORM	Declarative models, Session, relationships
0:55 - 1:15	Packaging	pyproject.toml, src/ layout, uv build
1:15 - 1:35	Docker Basics	Image vs container, Dockerfile, .dockerignore
1:35 - 2:00	Practice Problems	20 problems across Sections A-E

TIP

Instructor note: SQLAlchemy is the heaviest new concept today. If time is tight, prioritize Sections 1-4 of the ORM file (models through CRUD) and treat Section 7 (migrations) as a read-along rather than live-coded.

Prerequisites & Preparation

Requirements

Requirement	Why it's needed
Completed Day 8 (File I/O)	Context managers (`with`) reused for DB connections
Completed Day 9 (Error Handling)	try/except used around database operations
Completed Days 10-11 (OOP)	SQLAlchemy models ARE Python classes
Python 3.12 + uv (Day 1)	Environment for installing sqlalchemy
sqlalchemy package	Install with: uv add sqlalchemy

Instructor Pre-Session Checklist

#	Task
1	Confirm sqlalchemy is installed: pip show sqlalchemy
2	Delete any stale .db files from prior test runs in the working folder
3	Have Docker Desktop installed (or a screenshot ready) if live-demoing Section 5-8
4	Open DB Browser for SQLite (optional) to visually show table contents
5	Review the migrations note (Section 7) — keep it conceptual, not hands-on

Step-by-Step Instruction Guide

STEP 1 — SQLite Basics with sqlite3

What to Say

"Every app you've built so far forgets everything the moment it stops running. Today that changes. SQLite stores your entire database in a single file — no server, no setup — and Python has it built in. Let's connect, create a table, and insert our first row."

```
import sqlite3

connection = sqlite3.connect('tiffin_orders.db')
cursor = connection.cursor()
cursor.execute('CREATE TABLE customers (...)')
connection.commit()

# Output:
Table 'customers' created.
```

Walk through parameterized queries (? placeholders) immediately after — emphasize this is a security practice, not just style. Show `executemany()` for bulk inserts, then `sqlite3.Row` for dict-style column access.

Error	Cause	Fix
sqlite3.OperationalError: no such table	Table not created before querying	Run CREATE TABLE and commit() first
sqlite3.IntegrityError	Violates a constraint (e.g. duplicate PRIMARY KEY)	Check the value being inserted is unique
Changes don't appear in DB Browser	Forgot connection.commit()	Every write needs a commit() (except inside `with conn:`)

DONE WHEN

Students can run day25_01_sqlite_basics.py end-to-end and see every table, insert, JOIN, and aggregation query print matching output.

STEP 2 — SQLAlchemy ORM

What to Say

"Raw SQL strings work, but they live outside Python's type system — no autocomplete, no type checking. An ORM maps your tables to classes you already know how to write. Watch: this Author class IS the authors table."

```
class Author(Base):
    __tablename__ = 'authors'
    author_id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    books: Mapped[list['Book']] = relationship(back_populates='author')

# Output:
Tables created from models: authors, books
```

Emphasize the **Session** as a “shopping cart” for changes — objects added or modified inside a session aren't written to disk until `session.commit()`. Demo relationship traversal (`author.books`) — the ORM fetches related rows automatically, no manual JOIN needed for simple cases.

Error	Cause	Fix
sqlalchemy.exc.NoForeignKeysError	relationship() defined without a matching ForeignKey	Add ForeignKey('table.column') to the child's mapped_column
DetachedInstanceError	Accessing a relationship after the session closed	Access relationships while the `with Session(engine)` block is still open
AttributeError: object has no attribute	Typo in a column/relationship name	Double-check spelling matches the model definition exactly

DONE WHEN

Students can query, filter, update, delete, and aggregate (func.sum, func.count) entirely through Python objects — day25_02_sqlalchemy_orm.py runs clean with matching output for every example.

STEP 3 — Packaging with pyproject.toml

What to Say

"Right now, using your code means someone has to cd into your folder and run python somefile.py. pyproject.toml is what lets someone else pip install your-tool and just use it — from anywhere."

```
[project]
name = "grocery-tracker"
dependencies = ["sqlalchemy>=2.0", "typer>=0.9"]

[project.scripts]
grocery-tracker = "grocery_tracker.cli:app"

# Terminal:
$ uv build
Creates dist/*.tar.gz and dist/*.whl
```

Show the **src/ layout** and explain WHY it exists: it forces an actual install before import, catching "works on my machine" path bugs. This is exactly the layout the Capstone (taskcli) will use starting Day 26.

STEP 4 — Docker Basics

What to Say

"Packaging solves 'install my Python code.' Docker solves a bigger problem: 'run my code with the exact same OS and libraries, on any machine, every time.' An image is like a class — a container is an object created from it."

```
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir .
COPY src/ ./src/
CMD ["python", "-m", "grocery_tracker.cli"]

# Terminal:
$ docker build -t grocery-tracker:0.1.0 .
$ docker run grocery-tracker:0.1.0
```

OS	Check Docker is installed
macOS	docker --version (via Docker Desktop)
Windows	docker --version (via Docker Desktop + WSL2)
Linux	docker --version (via Docker Engine)

WATCH OUT

Docker is not installed in the course container used to build these materials — Section 6-8 of day25_03_packaging_docker.py is documentation-driven, generating real Dockerfile/.dockerignore/pyproject.toml files for students to inspect rather than requiring a live Docker daemon.

DONE WHEN

Students understand the image-vs-container distinction, can read every line of a basic Dockerfile, and know the four core docker CLI commands.

Session Wrap-Up

Student Checklist

#	Can you...
1	Connect to a SQLite file and run CREATE TABLE / INSERT / SELECT?
2	Explain why parameterized queries (? placeholders) matter?
3	Define a SQLAlchemy model with Mapped and mapped_column?
4	Perform CRUD through a Session and explain commit()?
5	Describe what pyproject.toml's three key sections do?
6	Explain the difference between a Docker image and a container?

Key Concepts Recap

Concept	One-line Summary
sqlite3 module	Built-in, file-based SQL database — no server needed
Parameterized queries	Use ? placeholders, never string-format SQL
SQLAlchemy ORM	Maps Python classes to database tables
Session	Unit of work — changes commit together as one transaction
relationship()	Auto-fetches related rows (e.g. author.books)
pyproject.toml	Declares project metadata, dependencies, entry points
uv build	Produces dist/*.whl — an installable package
Dockerfile	Recipe for building a reproducible image

Troubleshooting Notes

If a student's SQLAlchemy models raise import errors, first confirm the SQLAlchemy version with `pip show sqlalchemy` — this file uses 2.0-style `Mapped`/`mapped_column` syntax, which requires SQLAlchemy 2.0 or later.

Next Day Preview

Day 26 — Planning & Architecture kicks off the Capstone Project: writing the requirements spec, designing the ERD, and scaffolding the taskcli project folder with the same src/ layout and pyproject.toml pattern introduced today.

Quick Reference Card

Daily Workflow Commands

```
$ uv add sqlalchemy
$ python day25_01_sqlite_basics.py
$ python day25_02_sqlalchemy_orm.py
$ python day25_03_packaging_docker.py
$ uv build
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Toggle integrated terminal
Ctrl/Cmd + Shift + P	Command palette
F5	Run/debug current file
Ctrl/Cmd + K, V	Preview a Markdown file

Tool Summary

Tool	Purpose
sqlite3	Built-in raw SQL database access
sqlalchemy	ORM — Python classes mapped to tables
pyproject.toml	Project metadata & dependency declaration
uv build	Builds an installable .whl package
Docker	Reproducible, portable runtime environment